



Universidade de Brasília
Departamento de Ciência da Computação

Algoritmos de Busca

Professora: Eliza Gomes

E-mail: eliza.gomes@unb.br

Definição

- ▶ A busca nada mais é do que o ato de procurar por um elemento em um conjunto de dados;
- ▶ A operação de busca visa responder se determinado valor está ou não presente em um conjunto de elementos;
- ▶ No caso do item que se busca estar presente no conjunto de elementos, seus dados são retornados para o usuário;
- ▶ Toda busca é feita utilizando como base uma chave específica;
- ▶ A chave de busca é o “campo” do item utilizado para comparação;
 - ✓ É por meio dele que sabemos se dado elemento é o que buscamos.

Busca Sequencial ou Linear

Busca Sequencial ou Linear

- ▶ Esse algoritmo percorre o *array* que contém os dados desde a sua primeira posição até a última;
- ▶ Assume que os dados não estão ordenados, por isso a necessidade de percorrer o *array* do seu início até o seu fim;
- ▶ Para cada posição do *array*, o algoritmo compara se a posição atual do *array* é igual ao valor buscado;
- ▶ Se os valores forem iguais, o valor existe e o algoritmo retorna a sua posição no *array*.

Busca Sequencial ou Linear

	0	1	2	3	4	5	6
V	23	4	67	-8	54	90	21

Elemento procurado = 54

i=0	0	1	2	3	4	5	6	
	23	4	67	-8	54	90	21	Valor diferente: continua e busca
i=1	0	1	2	3	4	5	6	
	23	4	67	-8	54	90	21	Valor diferente: continua e busca
i=2	0	1	2	3	4	5	6	
	23	4	67	-8	54	90	21	Valor diferente: continua e busca
i=3	0	1	2	3	4	5	6	
	23	4	67	-8	54	90	21	Valor diferente: continua e busca
i=4	0	1	2	3	4	5	6	
	23	4	67	-8	54	90	21	Valor igual: termina a busca

Busca Sequencial ou Linear

```
int buscaLinear(int *v, int N, int elem){  
    int i;  
  
    for(i = 0; i < N; i++){  
        if(elem == v[i]){  
            return i; //elemento encontrado  
        }  
    }  
    return -1; //elemento não encontrado  
}
```

Busca Sequencial ou Linear Ordenada

- ▶ Assume que os dados estão ordenados;
- ▶ Assim, se o elemento procurado for menor do que o valor em determinada posição do *array*, tem-se a certeza de que ele não estará no restante do *array*;
- ▶ Isso evita a necessidade de percorrer o *array* do seu início até o seu fim;
- ▶ Ordenar um *array* também tem um custo;
- ▶ Esse custo é superior ao custo da busca sequencial no seu pior caso.

Busca Sequencial ou Linear Ordenada

	0	1	2	3	4	5	6
V	-8	4	21	23	54	67	90

Elemento procurado = 34

	0	1	2	3	4	5	6
i=0	-8	4	21	23	54	67	90

Valor diferente: continua e busca

	0	1	2	3	4	5	6
i=1	-8	4	21	23	54	67	90

Valor diferente: continua e busca

	0	1	2	3	4	5	6
i=2	-8	4	21	23	54	67	90

Valor diferente: continua e busca

	0	1	2	3	4	5	6
i=3	-8	4	21	23	54	67	90

Valor diferente: continua e busca

	0	1	2	3	4	5	6
i=4	-8	4	21	23	54	67	90

Valor é maior: elemento não existe

Busca Sequencial ou Linear Ordenada

```
int buscaLinearOrd(int *v, int N, int elem){
    int i;

    for(i = 0; i < N; i++){
        if(elem == v[i]){
            return i; //elemento encontrado
        }else{
            if(elem < v[i]){
                return -1; //para a busca
            }
        }
    }
    return -1; //elemento não encontrado
}
```

Busca Binária

Busca binária

- ▶ É um estratégia baseada na ideia de “dividir para conquistar”;
- ▶ A cada passo, esse algoritmo analisa o valor do meio do *array*;
- ▶ Caso esse valor seja igual ao elemento procurado, a busca termina;
- ▶ Do contrário, a busca continua na metade do *array* que condiz com o valor procurado.

Busca binária

	0	1	2	3	4	5	6	7	8	9
V	-8	-5	1	4	14	21	23	54	67	90

Elemento procurado = 4

	0	1	2	3	4	5	6	7	8	9
meio=4	-8	-5	1	4	14	21	23	54	67	90

Valor é menor: buscar no início

	0	1	2	3	4	5	6	7	8	9
meio=1	-8	-5	1	4	14	21	23	54	67	90

Valor é maior: buscar no final

	0	1	2	3	4	5	6	7	8	9
meio=2	-8	-5	1	4	14	21	23	54	67	90

Valor é maior: buscar no final

	0	1	2	3	4	5	6	7	8	9
meio=3	-8	-5	1	4	14	21	23	54	67	90

Valor é igual: terminar a busca

Busca binária

```
int buscaBinaria(int *v, int N, int elem){
    int i, inicio, meio, final;
    inicio = 0;
    final = N - 1;

    while (inicio <= final){
        meio = (inicio + final) / 2;
        if(elem < v[meio]){
            final = meio - 1; //busca na metade da esquerda
        }else{
            if(elem > v[meio]){
                inicio = meio + 1; //busca na metade da direita
            }else{
                return meio;
            }
        }
    }
    return -1; //elemento não encontrado
}
```

Busca em um *Array* de *Struct*

Busca em um *Array* de *Struct*

- ▶ Como fazer uma busca se no array estiver armazenado outro tipo de informação, como uma estrutura (*struct*)?
- ▶ Toda busca é feita utilizando como base uma chave específica;
- ▶ Essa chave é o “campo” utilizado para comparação;
- ▶ No caso de uma estrutura, a chave é o campo da *struct* usado para a comparação.

Busca em um *Array* de *Struct*

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void){
    struct aluno v[4] = {{2,"André",9.5,7.8,8.5},
                        {4,"Ricardo",7.5,8.7,6.8},
                        {1,"Pedro",9.7,6.7,8.4},
                        {3,"Ana",5.7,6.1,7.4}};
    int pos = buscaLinearNome(v, 4, "André");
    if(pos != -1){
        printf("Nome encontrado \n");
    }else{
        printf("Nome NÃO encontrado \n");
    }
    return 0;
}
```

```
struct aluno{
    int mat;
    char nome[30];
    float nota1, nota2, nota3;
};

int buscaLinearMatricula(struct aluno *v, int N, int elem){
    int i;
    for(i = 0; i < N; i++){
        if(elem == v[i].mat){
            return i;
        }
    }
    return -1;
}

int buscaLinearNome(struct aluno *v, int N, char* elem){
    int i;
    for(i = 0; i < N; i++){
        if(strcmp(elem, v[i].nome) == 0){
            return i;
        }
    }
    return -1;
}
```